LSTM Practice Questions: Solutions

I used the sigmoid $\sigma(z) = 1 / (1 + e^{-z})$, the tanh formula given in the instructions, and rounded intermediate values to 4 decimal places as asked.

Question 1: Single-cell forward pass

The plan is to find each gate's input z first, push it through the right activation, and then update the cell state before the hidden state, because h_t depends on C_t.

Given: $x_t = 0.5$, $h_{t-1} = 0.2$, $C_{t-1} = 0.4$.

Forget gate

$z = W_f \cdot x_t + U_f \cdot h_{t-1} + b_f = 0.7(0.5) + 0.3(0.2) + 0.1 = 0.35 + 0.06 + 0.1 = 0.51$

$f_t = \sigma(0.51) = 1 / (1 + e^{-0.51}) = 1 / (1 + 0.6005) = 1 / 1.6005 =$ **0.6248**

Input gate

$z = 0.6(0.5) + 0.4(0.2) + 0.0 = 0.30 + 0.08 = 0.38$

$i_t = \sigma(0.38) = 1 / (1 + e^{-0.38}) = 1 / (1 + 0.6839) = 1 / 1.6839 =$ **0.5939**

Candidate cell state

The candidate uses tanh rather than sigmoid, so it can be negative as well as positive.

$z = 0.9(0.5) + 0.2(0.2) + 0.0 = 0.45 + 0.04 = 0.49$

$\tilde{C}_t = \tanh(0.49) = (e^{0.49} - e^{-0.49}) / (e^{0.49} + e^{-0.49}) = (1.6323 - 0.6126) / (1.6323 + 0.6126) = 1.0197 / 2.2449 =$ **0.4542**

Cell state update

The forget gate decides how much of the old state stays, and the input gate decides how much of the candidate is added:

$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t = 0.6248(0.4) + 0.5939(0.4542) = 0.2499 + 0.2697 = 0.51967 \approx$ **0.5197**

(I kept the extra digits of each product here, 0.24992 and 0.26975, and rounded only the sum.)

Output gate

$z = 0.5(0.5) + 0.5(0.2) + 0.1 = 0.25 + 0.10 + 0.10 = 0.45$

$o_t = \sigma(0.45) = 1 / (1 + e^{-0.45}) = 1 / (1 + 0.6376) = 1 / 1.6376 =$ **0.6106**

Hidden state

$\tanh(C_t) = \tanh(0.5197) = (1.6815 - 0.5947) / (1.6815 + 0.5947) = 1.0868 / 2.2762 = 0.4775$

$h_t = o_t \cdot \tanh(C_t) = 0.6106 \times 0.4775 =$ **0.2916**

Without rounding along the way this comes out as 0.2915, so the small difference is only from the 4 d.p. rounding.

Summary

f_t	i_t	$\tilde{C}_t$	C_t	o_t	h_t
0.6248	0.5939	0.4542	0.5197	0.6106	0.2916

Question 2: Parameter count

Let m = 10 be the input size and n = 32 the hidden size. Each of the four gates (forget, input, candidate, output) has its own weights:

W (input to hidden): n × m = 32 × 10 = 320

U (hidden to hidden): n × n = 32 × 32 = 1024

b (one bias vector): n = 32

(a) One bias vector per gate

Per gate: 320 + 1024 + 32 = 1376

Total: 4 × 1376 = **5504 parameters**

(b) Two bias vectors per gate (PyTorch)

The only change is that each gate has 32 more bias values.

Per gate: 320 + 1024 + 32 + 32 = 1408

Total: 4 × 1408 = **5632 parameters**

As a check, the difference is 4 × 32 = 128, and 5504 + 128 = 5632. I also confirmed this by counting the parameters of nn.LSTM(10, 32) in PyTorch.

Question 3: Vanishing gradient through the cell state

(a) f = 0.9

k = 10: 0.9^{10} = **0.3487**

k = 50: 0.9^{50} = **0.0052** (more precisely 0.00515)

(b) k = 50

f = 0.5: 0.5^{50} = **8.88×10^{-16}**, which is effectively zero

f = 0.99: 0.99^{50} = **0.6050**

(c) What this means

A dependency 50 steps back can only be learned if enough gradient survives the trip, so with f = 0.5 the signal is gone, with f = 0.9 only about 0.5% is left, and with f = 0.99 about 60% is still there. In a vanilla RNN the gradient is repeatedly multiplied by weights and activation derivatives that it has no control over, so it usually shrinks quickly. In an LSTM the cell state is updated additively and the forget gate can be learned to stay close to 1, which keeps a much cleaner path for the gradient across long gaps.

Question 4: Electricity load forecasting

(a) Hourly records

3 years × 365 days × 24 hours = **26,280 records**

(b) Windows with stride 1

Each window needs 168 input hours plus 24 output hours, so 192 consecutive hours. A window can start anywhere as long as all 192 hours fit, which gives:

26,280 − 192 + 1 = **26,089 windows**

The +1 is there because a dataset of exactly 192 records would still give one window.

(c) Tensor shapes

Using (batch, time steps, features):

Input batch: **(64, 168, 3)**

Output batch: **(64, 24)**, since only load is predicted, for 24 hours. This is the same as (64, 24, 1) if the target keeps a feature dimension.

(d) Training windows and mini-batches

Training records: 0.70 × 26,280 = 18,396

The split is on records, so a training window must fit completely inside those records. Otherwise future test data would leak into training.

Training windows: 18,396 − 192 + 1 = **18,205**

Batches: 18,205 / 64 = 284.45, so 284 full batches and 1 partial batch of 29 windows

Mini-batches per epoch = **285**

(e) LSTM parameters

Same formula as Question 2, with m = 3, n = 64 and one bias vector per gate:

4 × (64 × 3 + 64 × 64 + 64) = 4 × (192 + 4096 + 64) = 4 × 4352 = **17,408 parameters**

Question 5: Predictive maintenance

(a) Min-max normalisation

$x_{norm} = (x - x_{min}) / (x_{max} - x_{min}) = (95 - 20) / (120 - 20) = 75 / 100 =$ **0.75**

(b) Prediction

0.83 is greater than the 0.5 threshold, so the model predicts **failure** within the next cycle.

(c) Confusion matrix and metrics

Starting from the 30 correct flags, the other counts follow from the 45 flags, the 40 actual failures and the 1,000 cycles:

TP = 30 (correct flags)

FP = 45 − 30 = 15 (flagged, but no failure)

FN = 40 − 30 = 10 (real failures that were missed)

TN = 1000 − 30 − 15 − 10 = 945

	Predicted failure	Predicted OK	Total
Actual failure	TP = 30	FN = 10	40
Actual OK	FP = 15	TN = 945	960
Total	45	955	1000

Precision = TP / (TP + FP) = 30 / 45 = **0.6667**

Recall = TP / (TP + FN) = 30 / 40 = **0.75**

F1 = 2 × P × R / (P + R) = 2(0.6667)(0.75) / (0.6667 + 0.75) = 1.0000 / 1.4167 = **0.7059**

Accuracy = (TP + TN) / total = (30 + 945) / 1000 = **0.975**

(d) Which metric matters

I think the team should prioritise **recall**. A missed failure (FN) can mean an unplanned breakdown and lost production, which usually costs far more than an unnecessary inspection (FP). The current recall of 0.75 means 1 in 4 failures is still missed. Precision should still be watched so that there are not so many false alarms that people stop trusting the model, and F1 is a useful way to track both.

Accuracy is misleading because the classes are very imbalanced, with only 4% of cycles being failures. A useless model that always says "no failure" would still get 960 / 1000 = 96% accuracy while catching none of the failures, so the 97.5% here looks better than it really is.